

Echo Loop - developer guide

Read `docs/GUIDE.md` (the engine and stack guide) first for the technology and workflow. This file is about this game: the rules, the code layout, and how the clever parts work (determinism, the generator, the bot, sharing).

- Branch `echoloop`, folder `EchoLoop`, app ID `com.maxmartin.echoloop`
- Saved data: `localStorage` key `echoloop.profile`
- Dev URL flags: `?seed=N`, `?replay=<text>`, and in dev builds `?speed=N`, `?autoplay=1`

1. The game

An arena (320 x 460 units, a 32 x 46 grid of 10-unit cells) with:

- **Orbs** to collect (about 5 to 9). Standing within 12 units of one collects it.
- **Laser barriers**: horizontal walls across the arena, each with one 4-cell **door**. The door is a deadly beam unless its **plate** (somewhere below it, same colour) has someone alive standing on it. Barriers divide the arena into bands; band 0 is the bottom, where you start. A game has 1 to 3 gates.
- **Drifters**: red spiked balls that bounce around inside one band, on a straight line forever.
- **Pulsars**: danger circles that switch on for 1.2 to 1.8 s in every 2.5 to 4.3 s, with a flicker half a second before.
- **Loops**: 20 seconds each (1200 ticks at 60 Hz). When a loop ends, the finger readings you made become a **ghost** that replays them exactly. In the next loop the ghosts and you are all in the arena.
- **Win** when, in one loop, every orb has been collected by you and the ghosts together.
- **Lose** after 8 loops (a rewarded ad gives 2 more). **Die** (touch anything deadly) and only that loop restarts: nothing else is lost.
- **Rewind now** ends a loop early. A ghost whose recording has run out stands still where it was.
- **Stars**: par is the number of loops the bot needed. 3 stars at par or better, 2 at one over, else 1.

Why it is a puzzle: nobody alone can pass a door and hold its plate. Loop 1 is "walk to the plate and stay"; loop 2 is "go through while your ghost holds it"; and so on. The ghosts' plates only ever help (an open door is never more dangerous), so an earlier ghost's recording stays valid when more ghosts are added. That monotonic property is what makes recorded ghosts safe, and the tests check it.

2. Determinism, the idea everything rests on

`stepRun(run, pointer)` is a pure function of the run state and the finger position. Two rules make it exactly reproducible:

1. **The finger is read only every 6 ticks** (10 times a second), quantised to whole numbers and held in between; what was read is recorded. A ghost is just those readings played back through the same code as the player. So a whole run is only `seed + the readings of each loop` (about 200 x 2 numbers per loop).

2. **The sim uses only + - * / %, comparisons,** `Math.sqrt`, `Math.round/floor` - never `sin`, `cos`, `pow`, `hypot`, `exp`, `log`. Those can differ in the last digit between browsers, and a recorded run must replay identically on a friend's phone. `tests/sim.test.ts` scans the sim folder and fails if one appears. Drifters bounce with a "triangle wave" formula (`fold` in `hazards.ts`), pulsars use integer arithmetic, and movement uses `sqrt` (which is exactly rounded by the standard).

Order of one tick (`run.ts`): latch finger readings (every 6th tick) -> move everyone toward their targets at 2.5 units/tick -> plates/doors (a gate is open while anyone alive is on its plate) -> danger (ghosts die and stop; the player dying ends the tick) -> orbs (ghosts first, in the order they were made, then the player) -> win / end-of-loop checks.

3. The generator (`generate.ts`)

`generateArena(seed)` tries `attempt = 0, 1, 2, ...`. Each attempt seeds its own RNG with `mix(seed, attempt)`, and `layoutArena` builds a candidate by rules: 1 to 3 gates (weights 15 / 40 / 45 %), bands of equal height, a door in a random column, a plate in the band below it (not near the start for gate 0), orbs spread out in every band (at least 3 in the top band, so every gate matters), and drifters and pulsars scaled by a random "heat". **Sanctuaries:** hazards keep clear of the start and every plate, because a ghost must be able to stand there for a whole loop.

`worthSolving` rejects layouts where the start or a plate is ever threatened, or an orb is dangerous more than half the time. Then the bot (below) must solve it; if it cannot, the next attempt is tried. After `maxAttempts` (24) a hazard-free layout is used as a last resort (never needed in the tests). The result is the same for the same seed on every device, and it comes with the bot's solution and `par`. Typical cost: about 15 ms per arena.

4. The bot (`plan.ts`)

The bot proves clearability and gives `par`. It plans on the grid, one **layer** per finger reading (200 layers per loop), because everything dangerous is known in advance:

1. `buildGrid`: for each layer, which cells are unsafe (drifters and pulsars painted onto the cells they threaten, with a small safety margin; barrier cells outside the doors are always unsafe). Door cells depend on when the ghosts open the door, so they are checked separately.
2. `ghostTimeline` plays the ghosts made so far (with nobody controlling a player) and records, tick by tick, whether each gate is open.
3. `search` is a forward sweep over (layer, cell): from the current cell, you may stay or step to any of the 8 neighbours (one cell per reading is always reachable at speed 2.5), if the destination is safe and any door it touches is open for the ticks it would be crossed. It stops at the earliest layer where a goal cell is reached.
4. `planLoop`: in loop k below the number of gates, go to plate k and stay there. Otherwise collect orbs the ghosts do not, nearest-in-time first.
5. The planned readings are **replayed through the real rules** (`verifyLoop`). Anything the planner got wrong (it cuts corners with margins) makes that arena get rejected, so the planner may be approximate but never wrong in what the game ships.

`par` is how many loops the bot used. A human who collects orbs on the way to the plates can beat it.

5. Sharing (`share.ts` and `engine/core/replay.ts`)

A run is `[1 (version), seed, loopCount, then per loop: length, x-steps..., y-steps...]`. Each loop's x readings and y readings are written as steps, and the engine's `encodeInts` writes the change of each step as a variable length number (a run of "no change" costs one token), as URL-safe base64. A bot solution is about 3 KB of text. `decodeReplay` rejects anything implausible (wrong version, lengths, values) and never throws. Opening a link generates that seed's arena, replays the recording through the real rules, and only if it **really wins** shows it ("A friend cleared arena N in L loops"). `stepReplay` handles loops that were ended early: when a recording runs out the loop ends there, exactly as it did live.

6. The flow (`index.ts`)

Phases: `menu`, `settings`, `playing`, `dead` (0.8 s, then the same loop restarts), `rewind` (0.95 s of scan lines, then the ghost joins), `winning` (1.4 s of confetti, then the result screen), `paused`, `result`, `lost` (watch an ad for +2 loops), `ad`, and two playback phases, `demo` and `viewer`, which step a recorded run at N ticks per frame. `stepLive` runs the sim (several ticks per update in dev with `?speed=N`), `settle` reacts to how it ended, and `handleEvent` turns sim events (orb, died, gate, ...) into effects and sounds. The sim never calls the UI. The interstitial rule (`shouldShowInterstitial`): not right after a rewarded ad, at most once a minute.

7. Drawing (`view/scene.ts`)

The scene redraws everything each frame from the run with `Graphics`: a static backdrop (grid, frame), danger zones, barriers with doors (a beam when shut, posts when open), plates joined to their door by a dotted line, orbs, drifters, and the movers with fading trails. Each ghost has its own colour (`ghostColor(loopIndex)`). Drifters and pulsars are drawn at the interpolated tick `run.tick - 1 + alpha`, using the same formulas as the sim. Effects (orb bursts, rings, death shake, the rewind scan lines, confetti) are particles that live only in the scene.

8. Tests

`npm run check` (typecheck + about 165 unit tests):

File	Covers
<code>tests/sim.test.ts</code>	movement and the 6-tick finger reading, recording, ghosts equal their recording to the bit, plates and gates, orbs in one loop, death and retry, rewind now, hazards, loop limits, no <code>sin/cos/pow</code> in the sim
<code>tests/generate.test.ts</code>	40 seeds: an arena exists, the bot's solution really wins, same seed = same arena, start and plates always safe, speed; sharing (round trip, garbage); stars, profile, ads pacing
<code>tests/replay.test.ts</code>	the engine's integer codec
<code>tests/architecture.test.ts</code>	the layer rules (from <code>main</code>)

`npm run e2e` plays every flow in a browser: menu, dragging, dying, the loop ending, pause, "Rewind now", the bot-driven run (`?autoplay=1&speed=30`) to three stars, share link round trip in a second page, a broken link, running out of loops and the rewarded ad, the demo, settings, a wide window. Screenshots go to `e2e/screenshots/` ; look at them.

9. How to change things

I want to...	Change
Make loops longer or shorter	<code>loopSeconds</code> in <code>sim/config.ts</code> (everything else follows)
Easier or harder arenas	<code>generator.minHeat/maxHeat</code> , the gate weights and orb counts in <code>layoutArena</code> , <code>drifter</code> and <code>pulsar</code> in the config
Change the loop limit or the ad bonus	<code>maxLoops</code> , <code>extraLoops</code>
Change the stars	<code>stars</code> in the config
A new hazard	a type in <code>types.ts</code> , its position/lethal rule in <code>hazards.ts</code> (pure, no <code>sin/cos</code>), painting it in <code>plan.ts</code> <code>buildGrid</code> , drawing it in <code>view/scene.ts</code> , tests
A new mechanic that helps ghosts	keep it monotonic if you can: more ghosts must never make an earlier ghost's recording unsafe, or the bot's reasoning breaks
Colours and look	<code>view/palette.ts</code> , <code>ui/styles.css</code>

10. Ship

```
npm run check && npm run build
npm run android:setup      # once (JDK 21 + Android SDK)
npm run android
```

Real AdMob: engine guide section 12 (this game uses a rewarded and an interstitial unit). A link for friends: engine guide section 10. **Not yet verified on a real phone:** how the drag feels (finger offset, speed 150 units/s), real ads and consent, vibration and sound, performance on a low-end device.